완전 탐색과 백트래킹

김태희

Contents

01 완전 탐색

02 순열

03 조합

04 부분 집합

05 완전 탐색 실습

06 백트래킹

완전 탐색

완전 탐색

- Exhaustive search
- Brute force
- 문제를 해결하기 위해 가능한 모든 경우의 수를 시도하는 알고리즘 기법
- 모든 경우의 수를 탐색하기 때문에 답을 놓칠 염려가 없지만, 경우의 수가 많아지면 비효율적일 수 있음
- 간단하고 직관적이기 때문에 상대적으로 빠른 시간 안에 알고리즘 설계 가능
- 일반적으로 경우의 수가 작을 때 유용
- 코딩테스트에서 문제를 풀 때, 우선 완전 검색으로 접근하여 해답을 도출한 후, 성능 개선을 위해 다른 알고리즘을 사용하고 해답을 확인하는 것이 바람직함

exhaustive

미국·영국 [ɪɡˈzɔːstɪv] 영국식

형용사

(하나도 빠뜨리는 것 없이) 철저한[완전한]

완전 탐색

- 많은 종류의 문제들이 특정 조건을 만족하는 경우나 요소를 찾는 것이다.
- 또한, 이들은 전형적으로 순열(permutation), 조합(combination), 그리고 부분집합(subset)과 같은 조합적 문제들 (Combinatorial Problems) 과 연관된다.
- 외판원 순회(TSP)
- 배낭 채우기
- 연구소
  - https://www.acmicpc.net/problem/14502
- 게리맨더링
  - https://www.acmicpc.net/problem/17471

순열

순열

- 서로 다른 것들 중 몇 개를 뽑아서 순서적으로 나열하는 것
- nPr = n * (n-1) * (n-2) * ⋯ * (n-r+1)
- nPn = n * (n-1) * (n-2) * ⋯ * 2 * 1
- 다수의 알고리즘 문제들은 순서화된 요소들의 집합에서 최선의 방법을 찾는 것과 관련

N	순열의 수
10	3628800
11	39916800
12	479001600
13	6227020800
14	87178291200

순열 생성

- {1,2,3} 중 3개를 뽑아 순서적으로 나열
- 각 자리마다 뽑는 행동
  - 1~3까지 자리에 넣을 수 선택하는 시도
  - 지금 선택하려는 수 앞자리의 수들에 있는지 체크 (중복 체크)
  - 중복되지 않으면 선택, 중복되면 다음수 시도
  - 다음자리수 뽑으러 고고씽!

순열 응용

- 주사위를 3번 던져서 나올 수 있는 모든 경우 출력

```
1 1 1
1 1 2
...
1 2 1
...
6 6 5
6 6 6
```

- 주사위를 3번 던져서 서로 다른 수를 순서적으로 나열 가능한 모든 경우 출력

```
1 2 3
1 2 4
...
1 3 2
...
3 2 1
...
6 5 3
6 5 4
```

조합

조합

- 서로 다른 n개의 원소 중 r개를 순서 없이 골라낸 것
- 예를 들어, {A, B, C}에서 두 개를 선택하는 조합은 {A, B}, {A, C}, {B, C}로 총 3가지
- 순열과 달리, {A, B}와 {B, A}는 동일한 조합으로 취급
- nCr = nCn-r

$$ {}_nC_r = \frac{{}_nP_r}{r!} = \frac{n!}{(n-r)!r!} $$

조합 생성

- {1,2,3} 중 2개를 순서 무관하게 선택
- 1-2 1-3
- 2-3

조합 응용

- 주사위를 3번 던져서 중복이 되는 경우를 제외하고 나올 수 있는 모든 경우

  - 1 1 2 와 중복 : 1 2 1, 2 1 1

```
1 1 1
1 1 2
...
1 1 6
1 2 2
...
5 6 6
6 6 6
```

- 주사위를 3번 던져서 모두 아래와 같은 중복되는 경우를 제외하고 다른 수가 나올 수 있는 모든 경우

  - 1 2 3 과 중복 : 1 3 2, 2 1 3, 2 3 1, 3 1 2

```
1 2 3
1 2 4
1 2 5
1 2 6
1 3 4
1 3 5
...
4 5 6
```

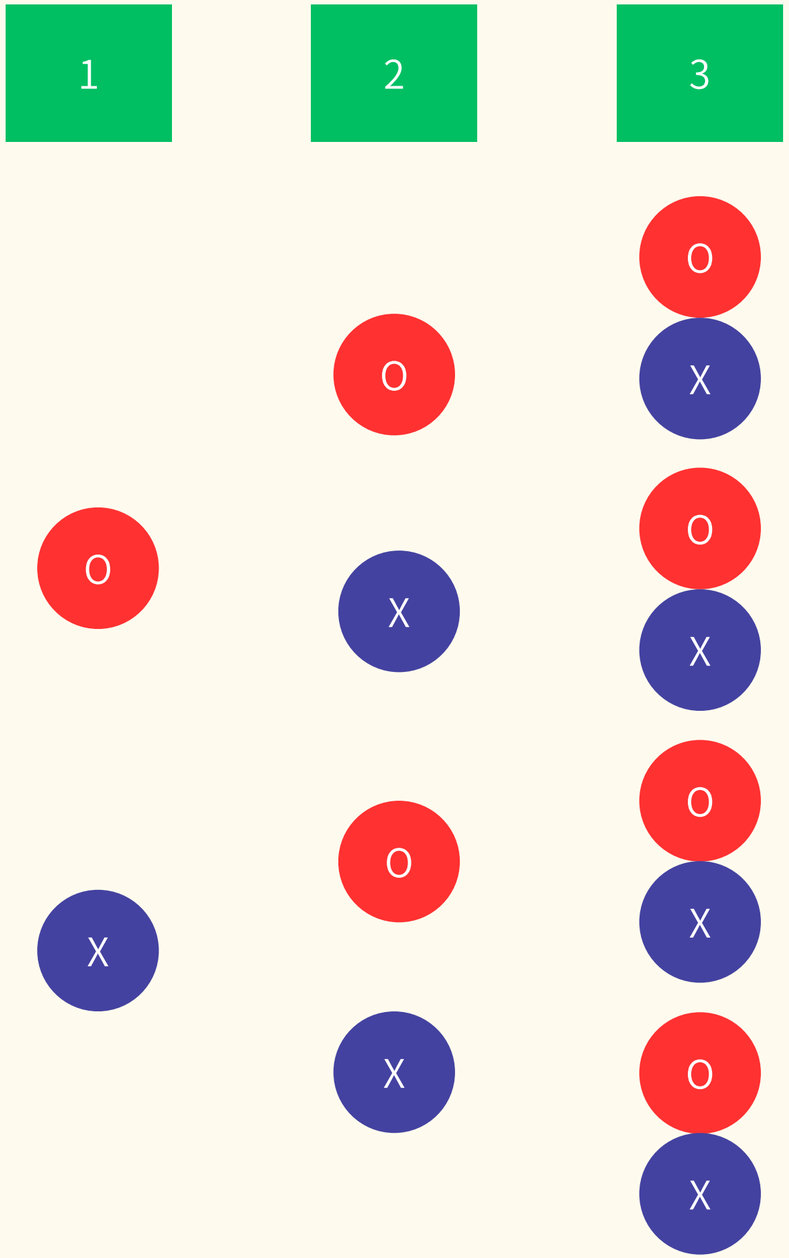
부분 집합

부분 집합

- 원래 집합의 모든 가능한 하위 집합
- 공집합과 자신도 부분 집합에 포함
- 예시: A, B의 부분 집합={},{A},{B},{A, B}
- 부분 집합은 원소를 선택하거나 선택하지 않는 방법으로 만들어지며,

  집합의 크기가 n일 때 공집합을 포함한 부분 집합의 개수는 2^n

  - 각 원소가 부분 집합에 포함, 포함되지 않는 두 가지 경우의 수



부분 집합 생성

- {1,2,3} 집합으로 만들 수 있는 모든 부분집합 생성

완전 탐색 실습

완전 탐색 실습

- 백설공주와 일곱난쟁이
  - https://www.acmicpc.net/problem/3040
  - 조합 : 9C7, 9C2. : nCr = nCn-r

- 캠프 준비
  - https://www.acmicpc.net/problem/16938

- 영재의 시험
  - https://www.acmicpc.net/problem/19949

백트래킹(BackTracking)

백트래킹(BackTracking)

- 기본 구현은 모든 가능한 해답을 탐색하여 문제를 해결하는 방법
- 모든 가능성은 하나의 트리처럼 구성할 수 있으며, 가지(선택지) 중에 해가 존재
- 트리의 루트(root) 노드에서 리프(leaf) 노드까지의 경로는 해답 후보가 되는데, 완전 탐색을 하여 그 해답 후보 중에서 해답을 찾을 수 있음
- 해답이 전혀 될 수 없는 노드도 모두 탐색하므로 비효율적임
- 해답의 후보를 점진적으로 구축하면서, 유효하지 않은 후보는 더 이상 확장하지 않고 포기(백트랙)하는 방식으로 동작

상태 공간 트리(state-space tree)

문제를 해결하기 위해 고려할 수 있는 모든 상태, 선택의 조합을 트리 형태로 표현한 것

각 노드는 현재까지의 결정을 나타내며, 자식 노드는 다음 단계에서 가능한 결정을 나타냄

백트래킹(BackTracking)

- 가지치기(pruning)
  - 상태 공간 트리 탐색 중, 유망하지 않은 경로를 미리 배제하여 탐색의 효율성을 높이는 방법
- 문제의 해를 찾는 과정에서 잘못된 선택을 했을 때
  - 이전 상태로 되돌아가 다른 선택지를 탐색하는 전략
- 재귀적으로 문제를 해결
- 조합 문제, 순열 문제, 그래프 탐색,

  퍼즐 문제(예: N-Queens 문제, 미로 찾기 등)에 많이 사용

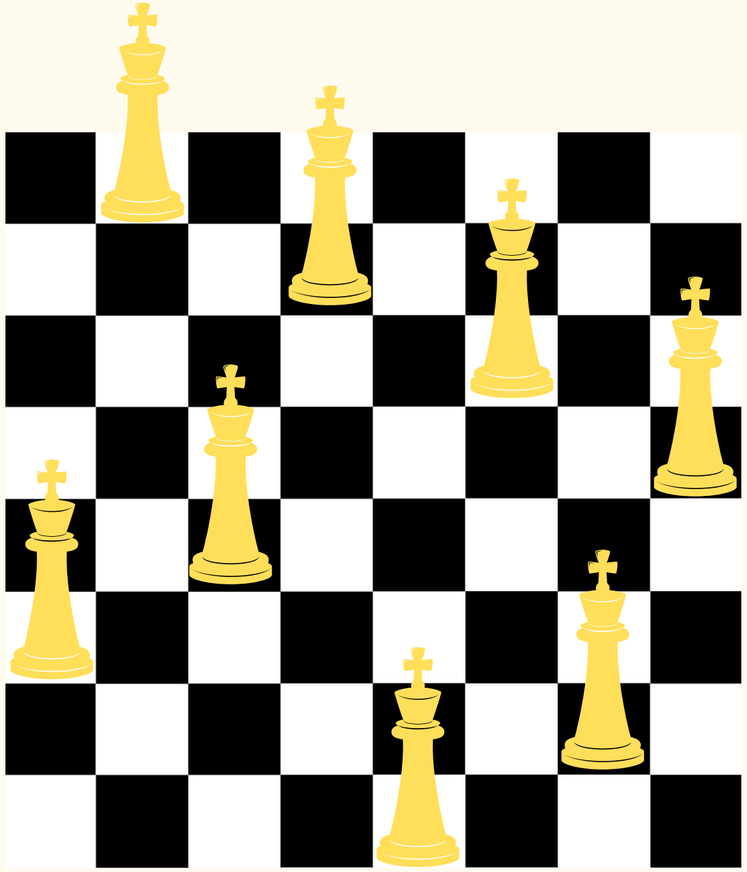
백트래킹의 기본 구조

```
def  backtrack(depth):
    if not 현재까지의 선택이 유효한지: return
    if 해 완성:
        해 처리
        return
    for 선택 in 선택의 후보들:
        현재 선택 저장
        backtrack(depth+1)
        현재 선택 되돌림
```

백트래킹 - NQueen

- N×N 서양 장기판에서 배치한 Queen들이 서로 양립 가능하도록 N개의 Queen을 배치하는 문제
- 퀸은 가로, 세로, 대각선 방향으로 이동하며 다른 퀸을 공격할 수 있음
- N개의 Queen을 모두 배치 가능한 경우의 수는?
  - 92개
- 완전 탐색으로 해결한다면?

$$ {}_{64}C_{8} = \frac{64!}{8!(64-8)!} = 4{,}426{,}165{,}368 $$



백트래킹 - 부분집합의 합

- 유한 개의 **자연수로** 이루어진 집합 { 5, 21,11,16,10 } 이 있을 때, 이 집합의 부분집합 중에서 원소를 모두 더했을 때 의 값이 21이 되는 경우의 수는?

  - 전체적인 코드 구조는?

  - 가지치기 요소?

백트래킹 응용

- NQueen
  - https://www.acmicpc.net/problem/9663
- 신기한 소수
  - https://www.acmicpc.net/problem/2023
- 스도쿠
  - https://www.acmicpc.net/problem/2239

Thank you!